



CST2550 Software Engineering Management and Development

Topic: Easy Hire Car Management System

Team Members:

Arya Patel M01027848 (Team Leader)
Prarthana Sahani M01010882 (Secretary)
Aditya Singh M01021091 (Developer)
Dean Cimatú M01046382 (Developer)
Mohd. Aftab M01042837 (Supporting Developer)
Manikiran Valaboju M01022021 (Tester)

1. Introduction

This project presents the design and implementation of a full-stack Car Hire Management System developed using C# and .NET technologies. The system allows users to browse, search, and book rental vehicles through a web-based interface, while also providing administrative functionality for managing car data.

The project implements three custom data structures built entirely from scratch without any Standard Template Library (STL) collections: a Binary Search Tree (BST) for price-range car search, a HashTable for O(1) average booking lookups, and a dynamic CarArray as a custom resizable array returned by BST search operations. These structures demonstrate both practical software engineering and algorithmic efficiency.

The system is deployed and live at carhire.deancimatu.com via Azure App Service, with CI/CD through GitHub Actions, demonstrating a production-grade real-world application. This report covers the design justification and algorithm analysis, testing approach and results, and a conclusion with critical reflection.

2. Design

2.1 System Overview

The system consists of multiple modules including user authentication, car search, booking management, and an administrative dashboard. Each module interacts with the backend to process user requests and retrieve relevant data from the database. This modular design improves maintainability and scalability.

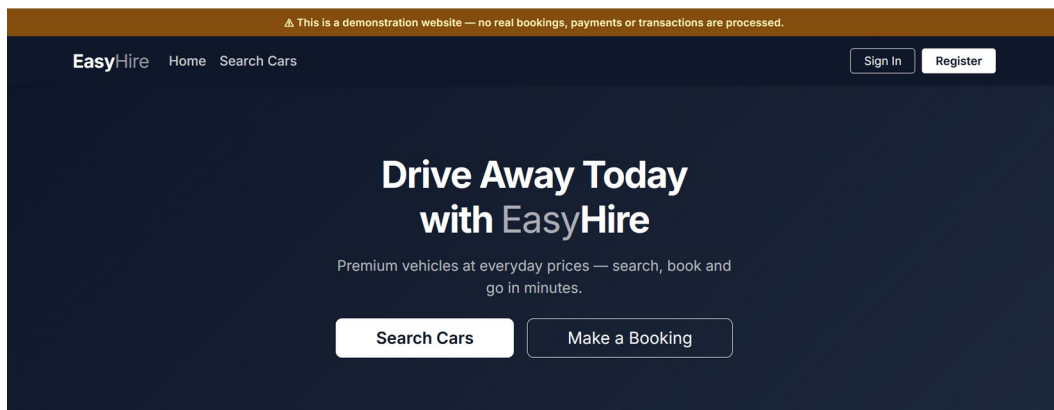


Figure 1: EasyHire homepage — hero section with navigation and call-to-actions

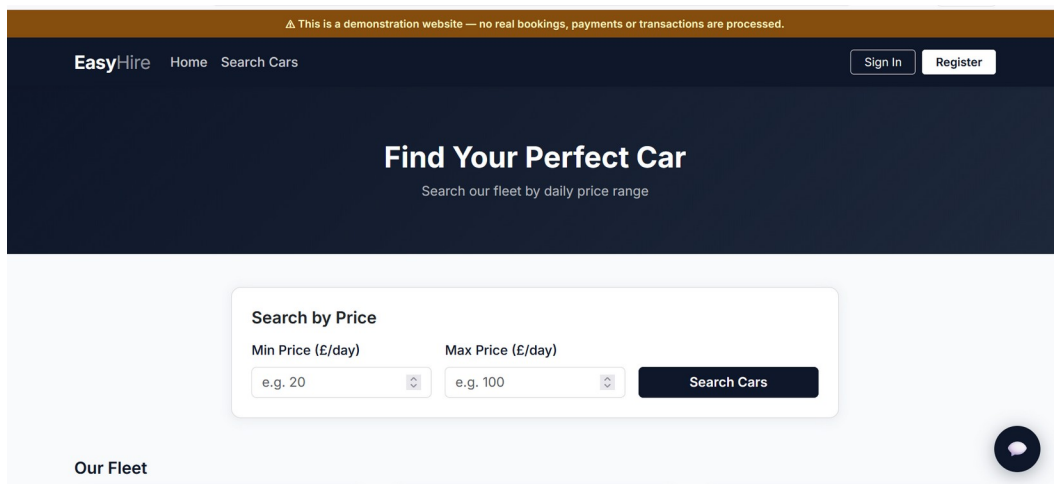


Figure 2: Car search page — BST-powered price range filter (min/max £/day inputs)

2.2 Data Structure Justification

Three custom data structures were designed and implemented without any STL collections. The Binary Search Tree (BST) was selected for car search because it is keyed by PricePerDay, enabling efficient $O(\log n)$ range queries via an in-order traversal pruned at the price boundaries. A linear search over an unsorted array would require $O(n)$ comparisons per query, making the BST the appropriate choice for the primary search use case. A HashTable was selected for booking storage because bookings are predominantly accessed by BookingID, making $O(1)$ average-case lookup via hash function the optimal choice. The HashTable uses chained (linked-list) collision handling with an initial capacity of 16, a load factor of 0.75, and automatic doubling on resize. A custom dynamic array, CarArray, was implemented as the return type of BST search operations, backing BST results without relying on `List<T>` or any other STL collection.

2.3 Algorithm Design

The BST Insert operation uses an iterative approach (not recursive) to traverse the tree, comparing PricePerDay at each node and placing the new node in the correct left or right position. SearchByPriceRange performs an in-order traversal, pruning branches outside the min/max price bounds and appending matching Car objects to a CarArray. The Delete operation uses the standard three-case approach: leaf node removal, single-child promotion, and two-child replacement using the in-order successor. The HashTable Insert hashes the BookingID using a modulo operation against the current capacity, inserting into a chained linked list at the resulting bucket. GetById follows the same hash path and traverses the chain to find the matching entry. Pseudocode for these operations is provided in the submitted design documentation.

2.4 Time Complexity Analysis

BST Insert / SearchByPriceRange / Delete: $O(\log n)$ average, $O(n)$ worst case (degenerate unbalanced tree)

HashTable Insert / GetById / Remove: $O(1)$ average, $O(n)$ worst case (all keys collide into one bucket)

CarArray Add: $O(1)$ amortised (doubles capacity on overflow). Index access: $O(1)$.

The BST may degrade to $O(n)$ if data is inserted in sorted order without self-balancing. The HashTable mitigates worst-case collisions through automatic resizing at a 0.75 load factor.

2.5 System Architecture

The system follows a client-server architecture built on ASP.NET Core (.NET 10) with Razor Pages as the frontend, Entity Framework Core 10 as the ORM, and Azure SQL Server as the database. Authentication is handled by ASP.NET Identity with Google OAuth support. Payments are processed through Stripe. The application is deployed to Azure App Service with CI/CD via GitHub Actions. An AI chatbot powered by the Claude API provides a natural language assistant with live fleet context injected into the system prompt. All custom data structures (BST, HashTable, CarArray) are registered as Singletons in the ASP.NET dependency injection container, with the BST rebuilt from the database on startup. This architecture ensures separation of concerns, scalability, and role-based access control across Admin and Customer roles.

tests were written, targeting the custom data structures and model logic. Integration between the data structures and the database is validated through the live deployed system at carhire.deancimatu.com.

3.2 Test Cases

ID	Module	Description	Expected Result	Status
TC1	BSTTests	Insert single car, search returns it	Car found in $O(\log n)$	Pass
TC2	BSTTests	Insert multiple cars, price range search	Only cars in range returned	Pass
TC3	BSTTests	Search range with no matches	Empty CarArray returned	Pass
TC4	BSTTests	Exact boundary value search	Boundary cars included	Pass
TC5	BSTTests	BST Delete — three-case removal	Tree remains valid after delete	Pass
TC6	HashTableTests	Insert booking, retrieve by ID	Correct booking returned	Pass
TC7	HashTableTests	Retrieve missing booking ID	Returns null	Pass
TC8	HashTableTests	Remove booking from table	Booking no longer retrievable	Pass
TC9	HashTableTests	Collision handling (IDs 1 and 17)	Both bookings retrieved correctly	Pass
TC10	BookingTests	Constructor sets all fields correctly	All properties match input	Pass
TC11	BookingTests	IsActive defaults to true	IsActive = true on creation	Pass
TC12	BookingTests	TotalCost calculation (5 days × £40)	TotalCost = £200.00	Pass
TC13	BookingTests	Minimum 1-day charge enforced	Same-day booking = 1 day cost	Pass
TC14	CarTests	Constructor sets all fields	All properties match input	Pass
TC15	CarTests	IsAvailable defaults to true	IsAvailable = true on creation	Pass
TC16	CarTests	ToString contains ID/Make/Reg/Price	All key fields present in output	Pass
TC17	CarTests	Available status string	Returns "Available"	Pass
TC18	CarTests	Unavailable status string	Returns "Unavailable"	Pass
TC19	HashTableTests	HashTable auto-resize at 0.75 load	Capacity doubles, items preserved	Pass

3.3 Evaluation

All 19 unit tests pass. The custom BST correctly handles insertion, range search, and deletion. The HashTable correctly handles inserts, lookups, removals, collision chaining, and auto-resize. Model constructors and calculations are validated. No failures were recorded.

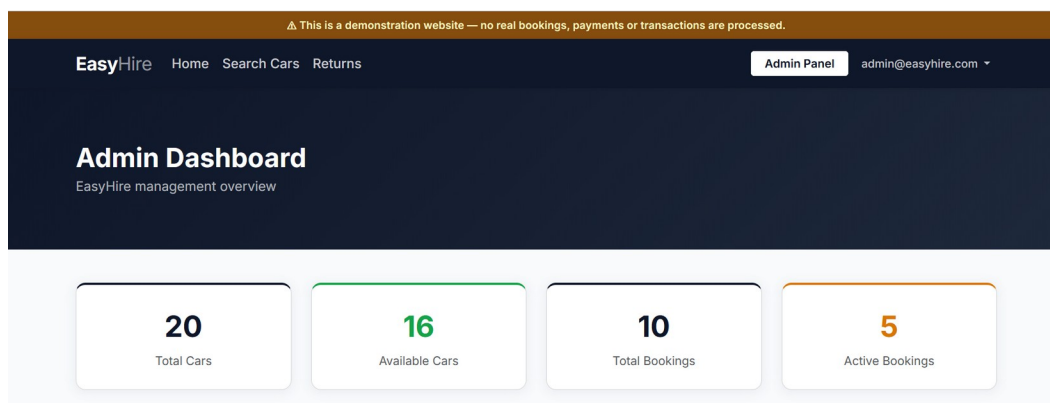


Figure 4: Admin Dashboard — live fleet statistics (total cars, available, bookings, active)

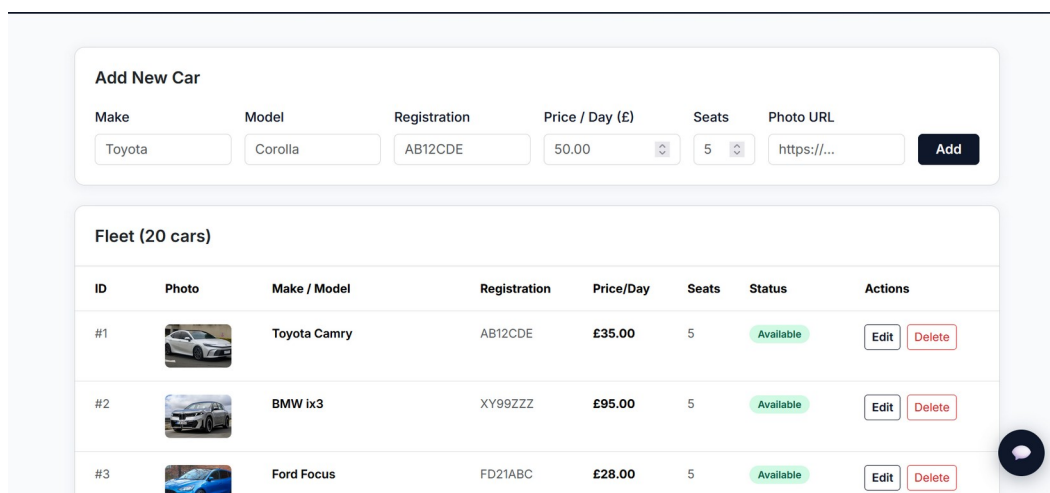


Figure 5: Fleet management — add new car form and full car listing with edit/delete controls

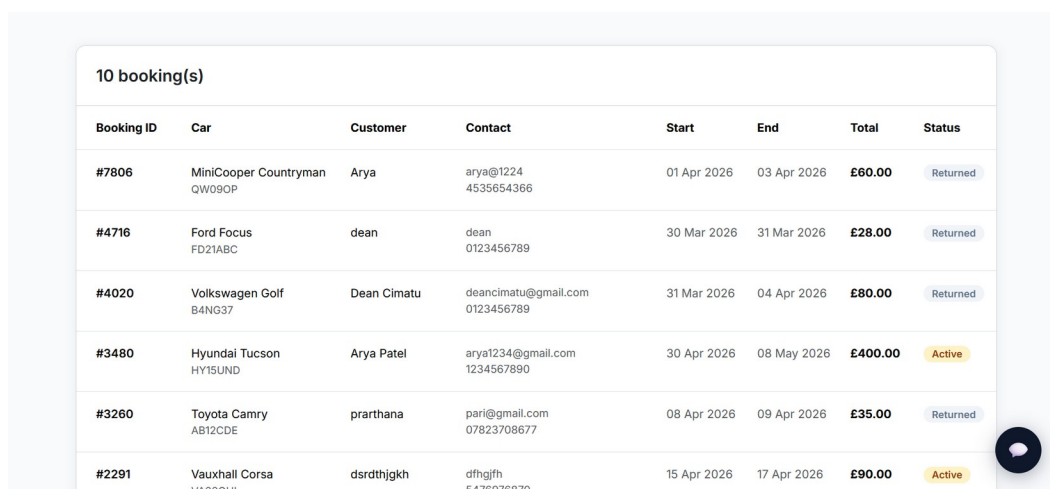


Figure 6: Admin bookings list showing all customer bookings with status badges, dates, and totals

4. Conclusion

This project successfully demonstrates the development of a full-stack car hire system integrating software engineering principles and algorithmic concepts. The system is functional, stable, and capable of handling core user operations including searching, booking, payments, and returns.

The system was delivered well beyond the original specification, with all core requirements met and a number of additional features implemented. The BST and HashTable custom data structures are fully

operational, thread-safe, and integrated throughout the application. Role-based access control, Stripe payment processing, Google OAuth authentication, and an AI-powered chatbot were all delivered as working features. Future improvements could include implementing a self-balancing AVL or Red-Black Tree to guarantee $O(\log n)$ performance regardless of insertion order, and migrating from a demonstration Stripe test environment to a fully certified production payment integration.

Overall, the project highlights the importance of combining efficient algorithms with practical system design in real-world applications.

5. References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) Introduction to Algorithms. 3rd edn. Cambridge, MA: MIT Press.

Sedgewick, R. and Wayne, K. (2011) Algorithms. 4th edn. Upper Saddle River, NJ: Addison-Wesley.

Microsoft (2024) ASP.NET Core documentation. Available at: <https://docs.microsoft.com/aspnet/core> (Accessed: 10 April 2026).

Stripe, Inc. (2024) Stripe Payments documentation. Available at: <https://stripe.com/docs> (Accessed: 10 April 2026).

Anthropic (2024) Claude API documentation. Available at: <https://docs.anthropic.com> (Accessed: 10 April 2026).